Mesh partitioning and node ordering for FESOM2/Kokkos: measurements on CPU and GPU, and a procedure for adopting a new partition

Technical report

13 August 2026 — Levante (128-core CPU nodes, 4×A100 GPU nodes; GH200 testbed)

TL;DR

- **GPU: a better mesh partition makes the model step 7–18 % faster** on three of the four meshes tested (3.9 % on the fourth). One factor dominates: the largest number of neighbour subdomains any process exchanges halo data with. METIS can minimise it (option `MINCONN`), but FESOM calls METIS through `PartGraphRecursive`, which ignores that option; calling `PartGraphKway` instead is a few-line change.
- **CPU: 4–8 % faster**, from two different factors: allowing METIS a few percent of imbalance in subdomain size (`UFACTOR=30` instead of the current 1), and using newer partitioning programs (KaMinPar, Mt-KaHyPar) in place of METIS. The best CPU and GPU partitions of the same mesh differ.
- Partitions built with identical settings but different random seeds already give slightly different solutions. We use the size of that scatter as the yardstick for every recommended partition, and **every recommended partition stays inside it** on temperature, salinity and sea-surface height (Section 6).
- **Cold starts must use the deterministic climatology fill** (`FESOM_IC_EXTRAP=det`). The routine that fills gaps in the initial temperature and salinity fields does so in an order that depends on the decomposition, so two partitions of one mesh start from *different oceans* — 27 salinity units apart on all four meshes here. That, not any property of the partitions, produced the eight model crashes and every accuracy objection in the first version of this report (Section 5). A new partition should still complete 3,000 steps at its target process count before use.
- **On DARS, simply regenerating the shipped partition with a different random seed is worth** −4.6 % — as much as any option we tested. This is specific to that mesh: on CORE2 at 864 processes and on fArc a fresh seed is 3–15 % *slower* than the shipped partition, so elsewhere the gains come from the options, not from luck.
- Renumbering the mesh nodes along a Hilbert curve pays only on CORE2 (−5.8 % together with repartitioning); the other meshes are already well ordered.

1. Summary

A FESOM2 run reads which mesh nodes belong to which process from the `dist_N` files that ship with the mesh. Replacing those files changes nothing else — no recompilation, no namelist change, no effect on any other configuration — so a better partition is the cheapest speed-up available to an existing setup. It is also one that has never been tuned: the partitioner has called METIS with the same options since its METIS-5 interface was written in 2015, and one of the four meshes here ships partitions that our build still reproduces byte for byte.

We generated several hundred partitions of four meshes — with METIS under settings FESOM does not currently use, with three other partitioning programs, and after two renumberings of the mesh nodes — and timed them against the shipped partitions on both backends of the C++/Kokkos port.

The shipped partitions cost the GPU backend up to 20 % of the model step. Most of that is recovered by a single METIS option, `MINCONN`, which minimises the largest number of neighbouring subdomains any process has and which no FESOM partition has ever had active, because `METIS_PartGraphRecursive` accepts the option and ignores it. The CPU backend gains 4–8 %,

from imbalance tolerance and from newer partitioning programs rather than from `MINCONN`; the two backends are best served by different partitions of the same mesh.

A partition change also perturbs the solution, and under a decomposition-independent initial condition that perturbation is no larger than the difference between two partitions built with the same settings and different random seeds — at every point measured here (Section 6). Eight partitions did make the model diverge during this work, but the cause was the initial condition rather than the partitions: the routine that fills gaps in the initial temperature and salinity fields does so in a decomposition-dependent order, and under the deterministic alternative all eight run to completion (Section 5). Running a new partition before trusting it remains sound practice.

Meshes: CORE2 (127k surface nodes, 1°), fArc (638k), DARS (3.16M), NG5 (7.40M, ≈5 km). All experiments are cold starts under JRA55 (1958) forcing from PHC climatology, at the largest time step at which a cold start of the given mesh is stable: 1800 s on CORE2, 900 s on fArc, 120 s on DARS, 180 s on NG5.

2. Candidates

METIS settings. Switching the METIS call from `PartGraphRecursive` to `PartGraphKway` makes three previously ignored options effective: the communication-volume objective, contiguity of the subdomains (`CONTIG`), and `MINCONN`, which minimises the largest number of neighbouring subdomains any subdomain has. We swept these together with `UFACTOR`, which sets how much the subdomains may differ in size (values 10, 30 and 100, i.e. 1, 3 and 10 %, against the 0.1 % FESOM uses now); with the node weight $w = a + \mathrm{nlev}$ for a between 0 and 100, which trades balancing the number of surface nodes per process against balancing the number of wet layers; with two-level hierarchical partitions; and with the METIS random seed. The Recursive call is not an oversight: `fort_part.c` documents that it produced the better partition on a test mesh, and lists which options it ignores. That comparison predates the GPU backend.

Other partitioning programs. KaHIP, KaMinPar and Mt-KaHyPar, each given the same weighted graph and the same sweep over a. FESOM does not interface to them; we exported the graph METIS is given, partitioned it externally, and read the result back, so the comparison is between partitions of one and the same graph.

Node renumbering. Renumbering the mesh nodes along a Hilbert space-filling curve, and alternatively by reverse Cuthill–McKee, so that nodes close in the mesh are close in memory. Unlike a partition, a renumbering changes the mesh files themselves, so every reference solution pinned to the old numbering must be re-established.

A second architecture. The best GPU settings were re-timed on DKRZ's GH200 nodes (dolpung), whose interconnect cannot register GPU memory; halos there are staged through pinned host buffers, which prices communication differently than the A100 fabric does.

3. Measurement procedure

Timing runs place all candidates in a single job allocation and interleave repetitions, so that node-to-node and time-of-day variation affects every candidate alike. Each run integrates 300 steps; we quote the best of N repetitions, as a percentage of the shipped partition's time per step. The mesh-definition files are verified byte-identical across candidates before the job starts, so the candidates differ in the decomposition only.

A timing result is accepted only after two further tests.

Stability. The candidate must complete 3,000 steps at the process count at which it will be used. Because the time step is set by the mesh, that is two months of model time on CORE2, one on fArc, but only four to six days on DARS and NG5 — a bound on the test, not a claim about long integrations. It exists because it fails in practice: a DARS partition that had won its timing comparison by 4.5 % diverged between step 150 and step 3,000 while the shipped partition ran cleanly.

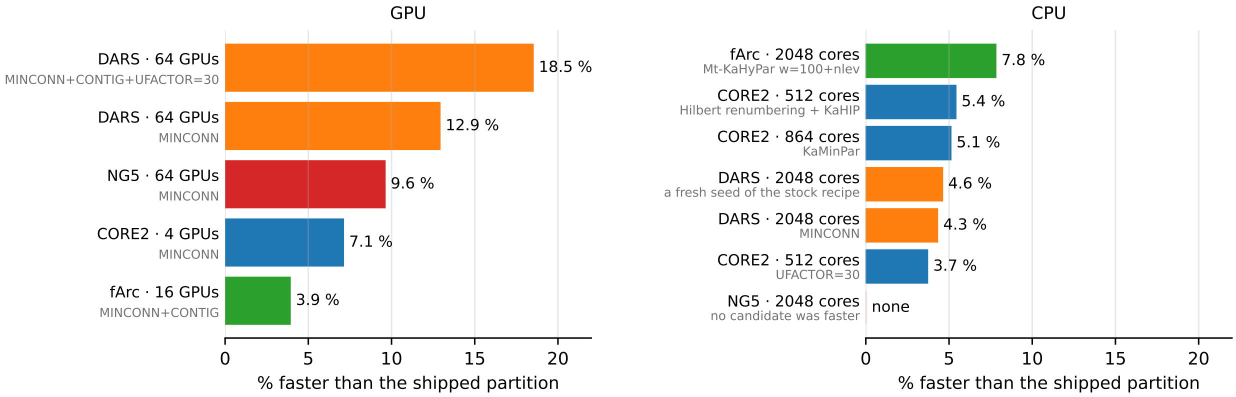

Figure 1: Best candidate partitions per mesh and process count, as a percentage faster than the shipped partition. Gray text under each label is the setting that produced the candidate. All candidates shown completed the 3,000-step run. Every candidate shown also lies inside the seed-partition scatter in temperature, salinity and sea-surface height (Section 6). Mesh colors recur in Figs. 2 and 3.

Size of the solution change. A different partition sums the same terms in a different order, and the resulting round-off differences grow at fronts, so a new partition cannot be asked to reproduce the reference solution bit for bit. The question is how large its difference from the reference is compared with the difference any other partition would produce. We therefore rebuild the shipped partition with three or more different METIS random seeds — checking in each case that the result really is a different partition — run twenty steps with each, and compare the candidate's rms difference from the reference in temperature, salinity and sea-surface height with the range those seed partitions span. Section 6 reports each candidate on that scale. Quantiles and maxima of the difference fields are recorded as well.

Twenty steps from a cold start measure whether a partition change is detectable at all; they say nothing about a multi-decade integration. Section 6 returns to this.

Partitions we recommend are packaged by a tool that refuses any `dist_N` without a recorded clean 3,000-step run at exactly N processes.

4. Speed

Figure 1 and Table 1 give the best candidate at each mesh and process count, with the setting that produced it. Every gain re-measured over 3,000 steps came back equal to or larger than its 300-step value, so the short timing runs do not overstate the effect.

mesh	processes	gain	setting	3,000 steps	solution change vs seed scatter
DARS	64 GPU	**−18.5** %	`MINCONN+CONTIG+UF=30`	clean	**within** (smallest T change of any leg)
DARS	64 GPU	**−12.9** %	`MINCONN`	clean	within (T, ssh); S at the edge
NG5	64 GPU	**−9.6** %	`MINCONN`	clean	**within** on T, S and ssh
CORE2	4 GPU	**−7.1** %	`MINCONN`	clean	below every seed partition
fArc	16 GPU	−3.9 %	`MINCONN+CONTIG`	clean	**within** (smallest T change of any leg)
fArc	2048 CPU	**−7.8** %	Mt-KaHyPar w=100+nlev	clean	within
CORE2	512 CPU	**−5.4** %	Hilbert renumbering + KaHIP	clean	within
CORE2	864 CPU	**−5.1** %	KaMinPar	clean	**at or below all five seed partitions**
DARS	2048 CPU	**−4.6** %	**a fresh seed of the stock recipe**	clean	within
DARS	2048 CPU	−4.3 %	`MINCONN`	clean	within
CORE2	512 CPU	−3.7 %	`UFACTOR=30`	clean	within
NG5	2048 CPU	none	—	—	—

Table 1: Best candidates per mesh and process count, relative to the shipped partition, at the time step given in Section 1. The last column gives the twenty-step rms difference from the reference where it exceeds what different METIS seeds produce, with the field responsible; Section 6 gives the numbers and what is known about their cause. On NG5 at 2,048 CPU processes no candidate was faster than the shipped partition.

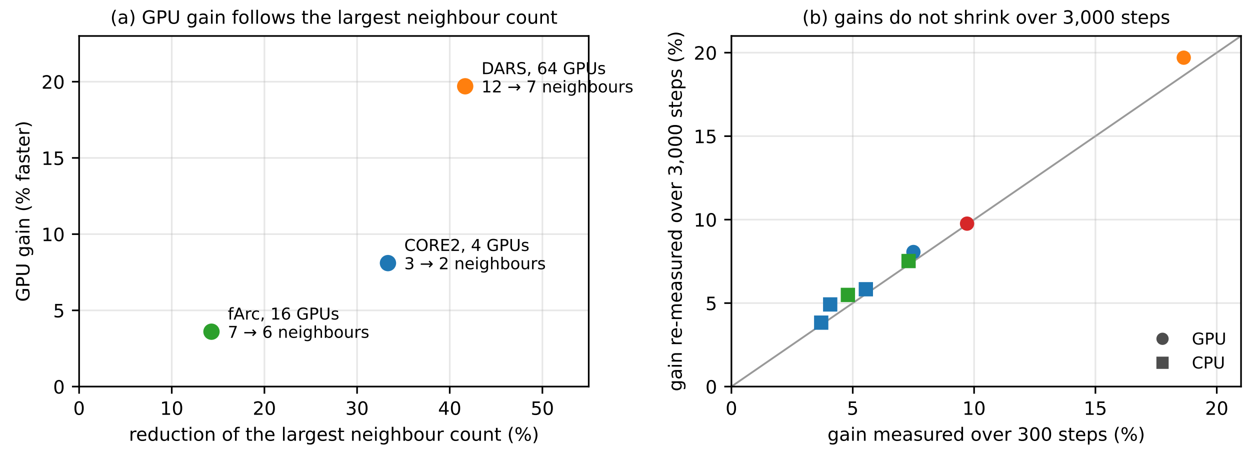

Figure 2: (a) GPU gain against the fractional reduction of the largest number of neighbour subdomains any process has, annotated with the counts before and after. (b) Gain measured over 300 steps against the same candidate's gain re-measured over 3,000 steps, for the eight candidates measured both ways; the line marks equality, circles are GPU points, squares CPU, and colors identify the mesh as in Fig. 1.

4.1. What the two backends pay for

Across the nine groups of timed candidates, the number of neighbour subdomains a process exchanges with is the only quantity computable from the partition that correlates positively with GPU step time. Every quantity measuring how much data is sent — edge cut, halo size, replicated elements — correlates negatively: on the GPU, candidates that send more data are faster if they send it to fewer neighbours. That is the behaviour of a machine paying per message rather than per byte, and `MINCONN` is the METIS objective that reduces the number of messages.

Across meshes the GPU gain follows the reduction in the *largest* neighbour count, not the mean (Fig. 2a): the mean falls by a similar fraction at all three points and so cannot order them, while the maximum ranges from −14 % (fArc, −3.6 % gain) to −42 % (DARS, −18.6 %). The maximum is the relevant statistic because each step waits for the slowest process. Section 4.3 shows that the coefficient of this relation is a property of the interconnect, not a constant.

On the CPU no quantity computable from the partition orders the candidates: for imbalance, edge cut and neighbour count alike, the rank correlation with step time straddles zero across the timed groups, so CPU candidates have to be timed. Two regularities survive. An external program won at every CPU point with 512 or more processes. And the same two options that gain five percentage points on DARS GPU when added to `MINCONN` — `CONTIG` and `UFACTOR=30` — lose one to two on DARS CPU.

4.2. Node renumbering pays only on CORE2

On CORE2, Hilbert renumbering is worth −1.2 to −2.4 % on CPU and −5.0 % on a single GPU node, and combines almost additively with repartitioning (−5.8 % at 512 processes). The other meshes gain nothing: 88 % of the nodes an element gathers from are already adjacent in memory, against 27.6 % on CORE2. Since renumbering forces every reference solution to be re-established (Section 2), it is worth taking only where that cost is acceptable; repartitioning carries no such cost.

4.3. The second architecture

On a single GH200 node the CORE2 gain reproduces at −8.9 % against −8.1 % on A100 (two independent jobs, agreeing within 0.6 percentage points), so it is not specific to A100 hardware. It does not survive the move to several nodes (Fig. 3). We registered a prediction before the job ran that DARS on 64 GH200 processes would exceed its A100 gain of −18.6 %; the measured effect is −1 to −3 %, and fArc on four nodes shows no effect at five repetitions. On this machine the per-message cost `MINCONN` removes therefore sits in the intra-node path, where halos are staged through host buffers, and not in the inter-node fabric. Multi-node gains have to be measured on each interconnect.

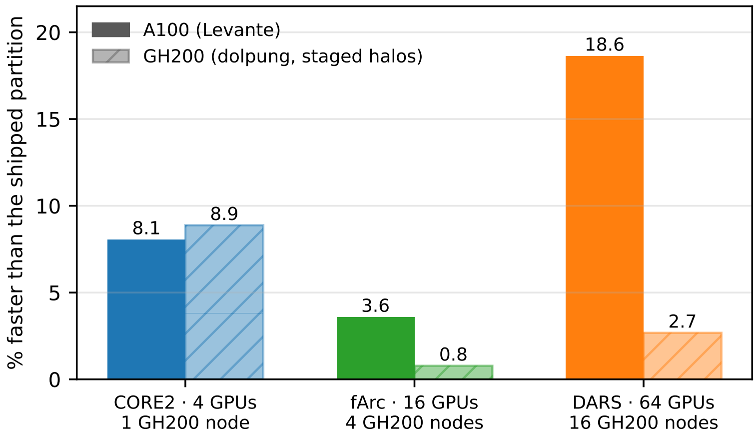

Figure 3: The three GPU settings re-timed on GH200 (hatched) against their A100 values (solid), with the GH200 node count under each pair. The GH200 values for fArc and DARS are medians, because the shipped partition was the noisiest configuration in those jobs and the best-of-N estimator is then biased; the fArc value is indistinguishable from zero at the measured noise level.

These jobs also showed that quoting the best of N repetitions is biased when the reference partition is the noisiest configuration, as it was there: best-of-4 made every fArc candidate look 0.8–2.3 % slower than the reference while the medians put them within 0.6 %. Where the scatter differs between configurations we report medians as well.

5. Why some repartitioned meshes appeared to crash the model

Eight of the partitions generated here made the model diverge: on NG5, `MINCONN` candidates at 2,048 and at 64 processes and a `UFACTOR=30` candidate at 64; on DARS, `MINCONN` candidates at 2,048; and, also on DARS at 2,048, one partition built with FESOM's current settings and nothing changed but the METIS random seed, which diverged within fourteen steps. The failures were local: salinity ran away at one location while the global diagnostics stayed normal.

None of them was a property of the partition. The cause is the routine that fills gaps in the initial temperature and salinity fields (`extrap_nod3D`, a port of upstream `gen_support.F90`). It fills each gap from whichever neighbouring values a process has already computed, running to local exhaustion between halo exchanges, so *which* water mass reaches a gap depends on where the subdomain boundaries fall. In marginal seas separated from the open ocean by a narrow strait — the Sea of Marmara, Gibraltar — an unlucky decomposition writes water of the wrong salinity across a single element. The resulting density step drives a constant pressure-gradient force, velocity ramps linearly from the first step, and the run dies on the CFL limit. Every scheme dies on such a partition; the stock solver is only the one that dies loudly.

The size of the effect, measured directly by writing the initial state for three partitions of each mesh and differencing them:

mesh (processes)	salinity	temperature	after one step (salinity)
CORE2 (512)	27.4	7.0 °C	27.4
fArc (2048)	26.0	9.0 °C	26.0
DARS (2048)	27.0	7.6 °C	27.0
NG5 (2048)	27.0	8.6 °C	27.0
all four, deterministic fill	**0**	**0**	10^{-14}–10^{-13}

Two partitions of the same mesh started about 27 salinity units apart — roughly the Black Sea/Mediterranean contrast — on every mesh, coarse and fine alike. With the deterministic fill (`FESOM_IC_EXTRAP=det`) the initial state is identical to the last bit across partitions, and *all eight partitions complete the 3,000-step run*, including the one built with FESOM's current settings, which is in fact among the fastest at its point.

Two practical consequences survive the correction. Short tests still do not substitute for long ones. And a cold-start comparison between two partitions is only meaningful under the deterministic fill: without it, the two runs are integrating different problems, which affects timings as well as solutions.

6. How much the solution changes

A new partition changes the solution. The question is how much, relative to what a user already accepts without knowing it: rebuilding the same partition with a different METIS seed changes the solution too. Every recommended candidate was re-measured under the deterministic initial-condition fill against at least three such seed partitions, twenty steps, field by field.

Every recommended candidate lies inside the seed scatter, on all three fields, at every point. Two of them — the fastest DARS 64-GPU candidate and the fastest fArc 16-GPU candidate — produce the *smallest* temperature difference of any run in their own comparison, seed partitions included. In every comparison the candidates and the seed partitions also follow an identical barotropic-solver iteration path, step for step.

This replaces the graded assessment of the first version of this report, in which four candidates exceeded the seed scatter and two were withdrawn. Those excursions were the decomposition-dependent initial condition of Section 5, not the model's response to the partition: the differences they were built from were 10^{-1}–10^{-2} in the affected fields, and the same comparisons under the deterministic fill give 10^{-7}–10^{-8}, five to six orders of magnitude smaller. Concretely:

first version	candidate	re-measured
withdrawn, "largest solution change measured"	DARS 64 GPU, `MINCONN+CONTIG+UF30`	within; smallest T change of any run
not recommended, on accuracy	fArc 16 GPU, `MINCONN+CONTIG`	within; smallest T change of any run
salinity rms +24 %	CORE2 864 CPU, KaMinPar	at or below all five seed partitions
sea-surface height +11 %, solver cause	DARS 64 GPU, `MINCONN`	within; identical iteration path
sea-surface height +8 %, cause unknown	NG5 64 GPU, `MINCONN`	within, all three fields

The one candidate still outside the seed scatter anywhere is NG5's `MINCONN+CONTIG+UFACTOR=30` (temperature 3.4 times the largest seed difference), which is not recommended at that point and is 3.8 percentage points slower than `MINCONN` there.

All numbers here come from twenty steps of a cold start. They show whether a partition change is detectable against the change a new seed already makes; they say nothing about a multi-decade integration. The cheap next test is to compare the model state after the 3,000-step run against the same seed partitions; the conclusive one is a multi-year pair of integrations.

7. Recommendations

For FESOM2 upstream, in order of value:

1. Call `METIS_PartGraphKway` in `fort_part.c`, so that `MINCONN`, `CONTIG` and the communication-volume objective reach METIS, and make the partitioner options run-time switches instead of compile-time constants.

2. Adopt the 3,000-step test together with the options: every new partition, whatever produced it, runs 3,000 steps at its target process count and the mesh's cold-start time step before production use; a failure is answered by rebuilding with a new seed.

3. Rebuild the isolated-node repair in `check_partitioning`, which currently takes a single neighbour as reassignment candidate, and update METIS to 5.2.1.

For runs on the machines measured here: use `MINCONN` on GPU, and decide `CONTIG` and `UFACTOR` by a timing run, since they help on some meshes and cost on others. On CPU use KaMinPar or Mt-KaHyPar with $w = 100 + \text{nlev}$, or `UFACTOR=30` if adding a partitioning program to the workflow is not worth it (at CORE2 512 the two are equal; at DARS 2,048 the program is 1.1 percentage points better and the only candidate that both ran 3,000 steps and stayed inside the seed scatter). Hilbert renumbering is worth taking on CORE2-class meshes where re-establishing the reference solutions is acceptable. Quantities computed from a partition are useful for designing candidates and useless for accepting them: acceptance is the 3,000-step run plus the comparison of Section 6.

8. Status

All measurements are complete. Four partitions — the CORE2 GPU and CPU winners, fArc 2,048 and DARS 2,048 — are packaged as mesh directories that carry their own run records; the DARS and NG5 64-GPU partitions can be packaged the same way. What remains is the upstream pull request.

Two questions are open. Whether an interconnect with working GPU-direct communication prices messages like the A100 fabric or like the GH200 test system decides how much of the GPU gain transfers to JUPITER, and only a measurement there will settle it. And for the three candidates whose temperature or salinity difference exceeds the seed scatter — among them the largest gain measured, −19.7 % on DARS — a long paired integration would show whether the difference matters at climate scale.

Job ids for every number quoted here, and the full record of the experiments, are in `docs/PARTITIONING_M11.md`; the operational summary is `docs/M11_RECOMMENDATION.md` and the complete table of timing runs is produced by `scripts/m11_harvest_races.py -best`.